User defined functions

Contents

- Subroutines. Modular programming
- Function. Function declaration, definition and call
- Function parameters. Formal and actual parameters. Arguments passing types
- Values return (by using *return* operator and by using parameters)
- Passing constants, variables, arrays, strings, function names to a function
- Parameters with modifier *const*
- Parameters with default values
- Functions with variable number of parameters
- Function overloading
- Function templates
- Recursive functions
- *main* function with parameters

Modular programming

- It is recommended to split complex tasks into subtasks and create a separate subroutines for each of them
- **Subroutine:**
  - separate independent program block with defined entry point and exit point;
  - set of instructions used to perform a particular task.
- Subroutines often are frequently used sets of instructions
- Subroutines:
  - improves program structure and reduces its size
  - makes code more readable, clear and reusable
  - makes code easier to maintain
  - reduce risk of errors
- Subroutines in C/C++ are **functions**

Define the code once and use it many times

Modules

- **Modules** – files with data and function groups
- Advantages:
  - Reusable code
  - May be compiled separately
  - Reduced compilation time
  - Easier to debug
  - Hidden implementation

- It is enough to know function <u>interface</u> (function name, parameters, return value type) to use it

Incapsulation

- Hiding implementation details
- Example:
  - Function *isdigit* – it is enough to know its prototype to use it correctly. Internal implementation is not important and is not needed to know
  - *isdigit* prototype:

int isdigit (int c);

Working with function

- Main function components:
  1) function **description**:
     a) function declaration
     b) function definition
  2) function **call**
- **Any function has to be described before it can be called!**

- **<u>Description</u>** should contain function name, formal parameters list and set of instructions
- **<u>Call</u>** should contain function name and actual parameters (arguments) list

Function declaration

- Declaration **syntax**:

```
type function_name([parameters_list]);
```

- Function **declaration** (**<u>prototype</u>**, **header**, **signature**) define its name, parameters list (may be empty) and return value type

- Function prototype should be placed in program text above the function call so that compiler could check function call correctness
- Function prototype is not required if function definition is placed above the call
- Parameters list should contain type and name for each parameter. Names are not required in prototype but recommended. Parameters are separated by comma

Function definition

- Definition **syntax**:

```
type function_name([parameters_list]) {
  function_body }
```

- Function definition should contain **header** un **body**

- **Function header in definition should match prototype** (parameters count, its type and order should match)
- Parameters list should contain type and name for each parameter. Names are required here. Parameters are separated by comma
- Body should contain instructions that need to be executed when calling a function

Function call

- Call **syntax**:

```
function_name([arguments_list])
```

- To call a function its name followed by round brackets should be specified
- Round brackets should contain arguments (parameter values) in the same order as in function prototype/definition header
- If parameters list in prototype/definition header is empty then round brackets in function call should be empty

Example 1

- **Function that prints line of 50 * characters on the screen**
- (program text is available in a separate file in moodle course)
- Running program example:

```
**************************************************
Data type        Range
**************************************************
char             -128...127
short            -32,768...32,767
int              platform dependent
long             -2,147,483,648...2,147,483,647
**************************************************

Press any key to continue . . .
```

Example 2

- Example without prototype
- **Function that prints line of 50 * characters on the screen**
- (program text is available in a separate file in moodle course)

- ....

- *Recommendation*: use function prototypes and place *main* function in the beginning of a program

Function parameters

- **Parameters** – way of data exchange between main program (or some other function) that calls a function and function itself
- **Parameters** – **values that are passed to function to make it more general**
- Parameters allow function to work with different values and perform different actions depending on passed values
- No limits for parameters count, but use as less as possible

Parameters count

- **Question:** how many parameters to use?
- **Answer**: should pass only data that is really needed

- A function that takes more than three or four parameters indicates one of two problems:
  - The function is doing too much. It should be split into several smaller functions with a smaller parameter set
  - There is another object hiding in. May need to create another object or data structure that includes these parameters

Function parameters (2)

- **Formal parameters** (**parameters**) – parameters that are specified in function header
- **Actual parameters** (**arguments)** – parameters that are specified in function call

Function call with parameters

- When calling a function:
  1. expressions in arguments are being evaluated;
  2. memory for formal parameters is being allocated according to its types;
  3. each parameter is being assigned an according argument value. At the same time:
     - data types are checked
     - if needed and if possible implicit conversion is made
     - error message is given in case of inappropriate type is used

Example 3

- Passing constants to a function
- **Function that prints on the screen line of equal characters. Character and repetition count are passed as constants (literals)**
- (program text is available in a separate file in moodle course)
- Running program example:

```
-----------------------------------------------
Data type        Range
===============================
char             -128...127
short            -32,768...32,767
int              platform dependent
long             -2,147,483,648...2,147,483,647
-----------------------------------------------

Press any key to continue . . .
```

Example 4

- Passing variables to a function
- **Function that prints on the screen line of equal characters. Character and repetition count is specified by user**
- (program text is available in a separate file in moodle course)
- Running program example:

```
Enter symbol: $
Enter symbol count: 30
$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$

Press any key to continue . . .
```

Return value

- ***return*** – operator that terminates the current function and returns the specified value (if any) to the caller
- **Syntax**:

```
return [expression];
```

- **Expression** – value that is defined as function result and which is being returned to the caller
- <u>Expression type</u> should be the same as type specified for return value in header or it should be possible to convert it implicitly or explicitly to a specified type!

Example 5

- Returning value from a function
- **Function takes speed in km/h as a parameter and return speed in m/s**
- (program text is available in a separate file in moodle course)
- Running program example:

```
Enter speed in km/h: 100
Speed in m/s: 27.7778

Press any key to continue . . .
```

Return value (2)

- If function returns a value by using *return* operator then **function call** may be considered **as expression** which value is equal to value returned by function. This expression may be used as any other expression, e.g. its value may be assigned to a variable

- **Value** that can be **returned** by using *return* operator always **is only one**. But it is possible to return a pointer to array, a structure or use another ways of returning a value

Example 6

- Elimination of unnecessary variables
- Example 5 modification. Demonstrates how expressions may be used without additional variables.
- ....
- *Note*: brevity or simplicity is programmer's choice

Arguments passing types

- **by value** (argument copy is passed to a function and if formal parameter is changed in function then actual parameter is not changed) (all previous examples with parameters)
- **by address** (argument address is passed to a function and if formal parameter is changed in function then actual parameter is changed also)
  - **by reference**
  - **by pointer**

Passing argument by value

```
...
void test(int);

int main()
{
   int a = 10;
   test(a);
   cout << a;        // a = 10
   return 0;
}
void test(int fa)
{
   fa = 15;
}
```

Passing argument by reference

```
...
void test(int&);    // parameter-reference

int main()
{
   int a = 10;
   test(a);
   cout << a;       // a = 15
   return 0;
}
void test(int& fa)
{
   fa = 15;
}
```

Passing argument by pointer

```
...
void test(int*);    // parameter-pointer

int main()
{
   int a = 10;
   test(&a);        // addess is being passed
   cout << a;       // a = 15
   return 0;
}
void test(int* fa)
{
   *fa = 15; // value is accessed by address
             // by using * (dereference operation)
}
```

Example 7

- Passing variable by pointer (+ value "return" through parameter)
- **Function takes value in feet and convert this value to metres**
- (program text is available in a separate file in moodle course)
- Running program example:

```
var = 15 feet
var = 4.572 metres

Press any key to continue . . .
```

Example 8

- Passing variable by reference (+ value "return" through parameter)
- **Function takes value in feet and convert this value to metres**
- (program text is available in a separate file in moodle course)
- Running program example:

```
var = 15 feet
var = 4.572 metres

Press any key to continue . . .
```

Example 9

- Passing variable by reference (+ several values "return" through parameters)
- **Function takes real number, defines whole part and decimal part and returns it**
- (program text is available in a separate file in moodle course)
- Running program example:

```
Enter real number: 15.123
Integer part is 15, fractional part is 0.123

Enter real number: 2.05
Integer part is 2, fractional part is 0.05

Enter real number: 0.0
Integer part is 0, fractional part is 0

Press any key to continue . . .
```

Arrays as function arguments

- **Arrays are always passed by address => function always works with original values, not with copies!**

- Address of array first element and array size should be passed. For C-strings it is not needed to pass the size
- To pass a multi-dimensional array need to pass all its sizes

Example 10

- Passing array to a function
- **1st function generates array of integers, 2nd function takes array, its size and prints array on the screen**
- (program text is available in a separate file in moodle course)
- Running program example:

```
91 40 753 451 299

Press any key to continue . . .
```

Example 11

- Passing array by pointer to a function
- **1st function generates array of integers, 2nd function takes array, its size and prints array on the screen**
- (program text is available in a separate file in moodle course)
- Running program example:

```
601 623 284 8 972

Press any key to continue . . .
```

Example 12

- Passing array by pointer to a function
- **Function takes array of values in feet, its size and converts values to metres**
- (program text is available in a separate file in moodle course)
- Running program example:

```
varray [ 0 ] = 4.572 metres
varray [ 1 ] = 10.5461 metres
varray [ 2 ] = 23.1343 metres
varray [ 3 ] = 7.22376 metres
varray [ 4 ] = 16.7945 metres

Press any key to continue . . .
```

Example 13

- Passing array by pointer to a function
- **Function takes array of values in feet, its size and converts values to metres**
- (program text is available in a separate file in moodle course)
- Running program example:

```
varray [ 0 ] = 4.572 metres
varray [ 1 ] = 10.5461 metres
varray [ 2 ] = 23.1343 metres
varray [ 3 ] = 7.22376 metres
varray [ 4 ] = 16.7945 metres

Press any key to continue . . .
```

Example 14

- Passing *C*-string by pointer to a function
- **Function gets string (*C*-string) and returns count of spaces in it**
- (program text is available in a separate file in moodle course)
- Running program example:

```
Enter string > This is example N14
Count of spaces is 3

Press any key to continue . . .
```

Example 15

- Passing *string* class string to a function (by value)
- **Function gets *string* class string and returns count of spaces in it**
- (program text is available in a separate file in moodle course)
- Running program example:

```
Enter string > This is example N15
Count of spaces is 3

Press any key to continue . . .
```

Example 16

- Passing *string* class string to a function (by value)
- **Function gets *string* class string and changes it**
- (program text is available in a separate file in moodle course)
- Running program example:

```
string in functions

Press any key to continue . . .
```

Example 16 (2)

```
...
void stringChange_test(string);
int main()
{
   string str;
   str = "string in functions";
   stringChange_test(str);// str remains unchanged
   cout << str;            // "string in functions"
   return 0;
}
void stringChange_test(string s)
{
   s = "Changed";
}
```

Example 17

- Passing *string* class string to a function (by reference)
- **Function gets *string* class string and changes it**
- (program text is available in a separate file in moodle course)
- Running program example:

```
Changed

Press any key to continue . . .
```

Example 17 (2)

```
...
void stringChange_test(string&);
int main()
{
   string str;
   str = "string in functions";
   stringChange_test(str);// stringChange_test function
                          // changed str string
   cout << str;           //"Changed"
   return 0;
}
void stringChange_test(string& s)
{
   s = "Changed";
}
```

Parameters with *const* modifier

- **Prevents parameter change in function**

parameter passing type doesn't matter

```
...
void constChange_test(const int&);
int main()
{
        int a = 10;
        constChange_test(a);
        cout << a;
        return 0;
}
void constChange_test(const int& fa)
{
        fa = 15;     // compilation error
}
```

Example 18

Function name as function argument

- Function may be called by pointer
- According type of pointer is being declared and function address is being assigned to it
- <u>Example</u>:

```
void f(int a) { /* ... */}  // function definition
void (*pf)(int); // declaration of pf pointer to a
                 // function
...
pf = &f;  // or pf = f;
      // address of f function is assigned to pf
      // pointer
pf(10);   // or (*pf)(10)
          // function f is called by pointer pf
```

Example 19

- Passing function name to a function
- **Function arranges array of integers (in ascending, descending order or by digit sum in its elements in ascending order). It takes array, its size as parameters and some function name as parameter that is used to compare elements**
- (program text is available in a separate file in moodle course)
- Running program example:

```
Enter element count in array: 5

-----generated array-----
634 136 852 769 790
------sorted array-----
136 634 769 790 852

Press any key to continue . . .
```

Parameters with default values

- "Optional parameters"
- **Should be last in parameters list** and may be omitted in function call
- **Constants, global variables, expressions** may be used **as** default **values**
- If argument is not specified in function call then all subsequent arguments also should not be specified
- Used when function arguments often have the same value or when value is not needed each time function is called

Example 20

https://www.programiz.com/cpp-programming/default-argument

https://www.learncpp.com/cpp-tutorial/default-arguments/

Variable number of parameters

- "Variadic functions"
- Formal parameter list ends with **...**
- <u>Prototype example:</u>

```
int printf(const char *, ...);
```

- <u>Call examples</u>:

```
printf("Enter first number");      //1 argument
printf("Enter %i number",d);       //2 arguments
printf("%d+%d=%d",a,b,sum);        //4 arguments
```

- Info about actual parameters type and count is taken from explicitly declared parameter value

https://www.tutorialspoint.com/variable-number-of-arguments-in-cplusplus

https://en.cppreference.com/w/cpp/utility/variadic

Example (1)

```
...
int sum(int n, ...); // prototype
void main(){
    int result = 0;
    result = sum(4, 7, 1, 10, 2);
    cout << "Sum = " << result << endl;
}
int sum(int n, ...) // n - count of unknown parameters
{   int *p = &n;
    p++;
    int res = 0;
    for(int i = 1; i <= n; i++){
        res += (*p);
        p++;
    }
    return res; }
```

Example (2)

```
...
int sum(int a1, ...); // prototype
void main(){
    int result = 0;
    result = sum(7, 1, 10, 2, 0); // 0 in the end - as end
                                  // of list indicator
cout << "Sum = " << result << endl;
}
int sum(int a1, ...) // a1 - first number to sum
{   int *p = &a1;
    int res = 0;
     while (*p) { // while there is a parameter-value and
                  // it is not 0
        res += (*p);
        p++;
    }
    return res; }
```

Function overloading

- **Overloaded functions** – multiple functions with the same name but different parameter types and/or its count
- Compiler defines which function should be called by actual parameters count and/or its type
- Worth to overload functions that perform same or similar actions but for example with data of different types
- Make development simpler as there is no need to remember many function names

Example 21

- Function overloading
- **Function defines maximum of two values. 4 functions are defined with the same name and different type of parameters**
- (program text is available in a separate file in moodle course)
- Running program example:

```
Enter 2 integer values: 3  5
max(int,int) is 5
Enter 2 real values: 2.5  0
max(float,float) is 2.5
Enter 2 symbols: a  z
max(char,char) is z
Enter 2 strings (max 9 symbols each): abc  ax
max(string,string) is ax


Press any key to continue . . . _
```

Rules for overloaded functions

- Should have same scope
- May have optional parameters (with default values) but same parameter values should be equal (if these functions don't differ in other parameters)
- Functions **can't be overloaded** if parameters differ only in *const* modifier or reference presence and also if functions differ only in return value type

Function templates

- Function template allows to define algorithm that can be applied to data of different types and information about specific type is passed to a function during execution
- Function template **syntax**:

```
template <typename type_name>
function_header
{ function_body }
```

instead of *typename* word *class* may be used

- **`type_name`** can be any. For example, **`T`**, **`Type`**, **`myType`**, **`someType`** etc.
- Function header may contain parameters of any type and of some type that is specified as template argument behind *typename*

Example 22

- Function template
- **Function template defines maximum of two values (of any type where comparison operation '>' can be used)**
- (program text is available in a separate file in moodle course)
- Running program example:

```
Enter 2 integer values: 3  5
maxT() is 5
Enter 2 real values: 2.5  0
maxT() is 2.5
Enter 2 symbols: a  z
maxT() is z
Enter 2 strings (max 9 symbols each): abc  ax
maxT() is abc


Press any key to continue . . .
```

Function templates (2)

- Can be used for data types where all functions and operations that are used in template are defined

Example 23

- Function template with multiple parameters of template type and basic type
- **Function template takes array, its size and some value as parameters. Function search for specified value in array and returns found value index or -1 if value is not found**
- (program text is available in a separate file in moodle course)
- Running program example :

```
'c'in chrArray: index = 2
 7 in intArray: index = -1
 4 in dubArray: index = -1

Press any key to continue . . .
```

Recursive functions

- **Recursive function** is a function that calls itself – explicit recursion
- Implicit recursion – two or more functions call each other
- If function calls itself then copies of parameters values are made in stack and then control is passed to function body
- Each recursion function **must have termination condition**
- Examples – calculating factorial, raising to a power, working with data structures, e.g. binary trees, quicksort (Hoare algorithm), etc.

Example 24

- Using recursion
- **Recursive function to calculate number factorial**
- (program text is available in a separate file in moodle course)
- Running program example:

```
5
factorial = 120

Press any key to continue . . .
```

#	Action	Argument or returned value
1	Call	5
2	Call	4
3	Call	3
4	Call	2
5	Call	1
5	Value return	1
4	Value return	2
3	Value return	6
2	Value return	24
1	Value return	120

Recursive functions (2)

- "+":
  - The code may be easier to write
  - To solve such problems which are naturally recursive such as tower of Hanoi
  - Reduce unnecessary calling of function
  - Extremely useful when applying the same solution
  - Recursion reduce the length of code
  - It is very useful in solving the data structure problem

Recursive functions (3)

- "–":
  - Recursive functions are generally slower than non-recursive functions
  - It may require a lot of memory space to hold intermediate results on the system stacks
  - Hard to analyze or understand the code
  - It is not more efficient in terms of space and time complexity
  - The computer may run out of memory if the recursive calls are not properly checked

main function

- Can return value to system that calls it (OS) and take parameters from environment
- *main* function **prototype**:

```
int main (int argc, char* argv[]);
```

- *argc* – parameters count (including application name)
- *argv* – pointer to array of pointers of type *char**. Element with index 0 is application name
- Parameters are space separated list

Example 25

- Additionally:
  - http://www.cplusplus.com/doc/tutorial/functions/
  - http://www.cplusplus.com/doc/tutorial/functions2/
  - http://www.cplusplus.com/articles/D2N36Up4/
  - etc.